Organização: Checa AI

Versão	1.0, 14/10/2026
Período coberto	10/09/2026 (primeiro commit do modelo) a 14/10/2026 (entrega)
Desafio	First Challenge: em um mundo com excesso de informação, como distinguir fatos, evidências e opiniões? A IA pode apoiar a investigação da confiabilidade das informações, fortalecendo o pensamento crítico em vez de substituí-lo.
Repositórios	`thiago-cg/Checa-AI` (aplicação e entrega), `thiago-cg/FakenewsBR` (dataset e modelo), `mariadenis/crosscheckbr-site` (site)

Fontes deste documento. Os papéis e a forma de trabalho foram informados pela equipe. As frentes de cada integrante vêm do documento de Guiding Questions (`entrega/guiding_questions/`) e do histórico do GitHub (pull requests e commits até 14/10/2026).

Sumário

1. Introdução
2. Papéis
3. Integrantes e frentes de trabalho
4. Forma de trabalho
5. Linha do tempo
6. Artefatos de acompanhamento
7. Pontos a melhorar no processo

---

1. Introdução

Este documento registra como a equipe do Checa AI se organizou: os papéis do Scrum, quem respondeu por cada frente do projeto, a forma de trabalho adotada e os artefatos que acompanharam o andamento. O produto é um assistente de checagem de fatos em português que estima a propensão de uma notícia ser desinformação, com as fontes e o passo a passo da análise.

---

2. Papéis

Papel	Quem	Responsabilidades
Product Owner	Mentores do programa	Propor o desafio, orientar a direção do produto e avaliar as entregas.
Scrum Master	Daltro Oliveira Vinuto	Facilitar as reuniões da equipe e acompanhar o andamento e os impedimentos.

Papel	Quem	Responsabilidades
Time de desenvolvimento	Thiago Correia Gonzaga, Maria Eduarda Denis Duarte Marques, Guilherme Silva Dutra, Gabriella Oliveira de Souza Dias, Eliane Orlandin do Carmo	Pesquisar, construir, testar e documentar o produto, dividindo o trabalho por frentes (seção 3).

As decisões de produto do dia a dia (por exemplo, apresentar a resposta como "propensão de ser fake news" e não como veredito) foram tomadas pela equipe e ficaram registradas em `docs/PROXIMOS_PASSOS.md`.

---

3. Integrantes e frentes de trabalho

Integrante	Frente principal	Guiding Questions	Contribuições registradas no GitHub
Thiago Correia Gonzaga	Núcleo da aplicação e modelo próprio	2, 2.1, 7, 7.1 e 10	Agente de busca, juiz e avaliador por notícia, telemetria e avaliação (`eval/`), núcleo de decisão, base primeiro (E1), relevância temporal (E4), auditoria de agentes e prompts; no FakenewsBR, o pipeline do BERTimbau e a versão v6 do dataset. 11 pull requests.
Maria Eduarda Denis Duarte Marques	Requisitos, produto e integrações	5 e 9	Engenharia de requisitos e de produto de IA, site do projeto, linguagem de "propensão", modelo de linguagem local, confiabilidade das fontes, base histórica de checagens, simulação de usuário, Fases D e F, Google Fact Check API, entrega dos requisitos. 12 pull requests.
Guilherme Silva Dutra	Organização da entrega e do repositório do modelo	1, 5.1, 5.2 e 8	Pasta `entrega/`, dataset e scripts de treinamento do modelo, relatório técnico do modelo; no FakenewsBR, organização do repositório e testes.
Gabriella Oliveira de Souza Dias	Pesquisa	3 e 3.1	Documento de Guiding Questions (`entrega/guiding_questions/`).
Eliane Orlandin do Carmo	Análise exploratória dos dados	4	EDA do dataset v6, com relatórios, gráficos e o pipeline (`entrega/EDA/`).
Daltro Oliveira Vinuto	Scrum Master e processo	11	—

As perguntas 6, 6.1 e 12 (transparência da avaliação, apresentação sem enviesar e aplicações) foram respondidas por toda a equipe.

---

4. Forma de trabalho

Prática	Como foi feita
Ciclos	Sem sprints formais. O trabalho foi organizado por fases de entrega (A a G, em `docs/PROXIMOS_PASSOS.md`), cada uma com itens, responsável e situação.
Reuniões	Reuniões curtas e frequentes de alinhamento (daily).

Prática	Como foi feita
Fluxo no código	Uma branch por tarefa, pull request para a `main` com o que mudou, as medições e o que ficou sem validar; os entregáveis do desafio entram pela branch `entrega`.
Critério de pronto	Testes automatizados passando; avaliação de decisão antes e depois registrada no pull request e em `eval/ITERACOES.md`; nenhum erro grave novo; commits por tema; o que ficou sem validar escrito no pull request.
Decisões	Registradas no próprio repositório: planos aprovados em `docs/superpowers/plans/`, decisões e pendências em `docs/PROXIMOS_PASSOS.md` e na seção de pendências dos requisitos.

---

5. Linha do tempo

Período	Marco	Pull requests
10/09 a 23/09	Pipeline do BERTimbau e dataset v6 (FakenewsBR); primeira versão do robô; juiz por notícia	#1
28/09 a 30/09	Revisão de arquitetura, agente de busca, telemetria e avaliação, requisitos e produto de IA, site	#2 a #9
05/10 a 07/10	Linguagem de "propensão", confiabilidade das fontes, base histórica de checagens, base primeiro (E1), avaliador por notícia	#10 a #15
09/10 a 10/10	Simulação de usuário, relevância temporal (E4)	#16 a #18
13/10 a 14/10	Auditoria de agentes, Fases D e F, Google Fact Check API; entrega: Guiding Questions, EDA, dataset e treinamento, relatório técnico, requisitos	#19 a #25

---

6. Artefatos de acompanhamento

Artefato	Onde está	Para que serve
Fases e pendências	`docs/PROXIMOS_PASSOS.md`	Lista do que fazer, por fase, com situação e decisões
Registro de iterações	`eval/ITERACOES.md`	Hipótese, mudança e medição de cada rodada de melhoria
Planos de trabalho	`docs/superpowers/plans/`	Plano de cada fase, aprovado antes da implementação
Requisitos	`entrega/requisitos/`	Requisitos, status no código e pendências de decisão
Pull requests	GitHub, `thiago-cg/Checa-AI`	O que cada mudança fez, medições e revisão

---

7. Pontos a melhorar no processo

Ponto	Sugestão
Sem backlog priorizado em ferramenta própria (o repositório não tem issues nem quadro de tarefas)	Usar issues e um quadro do GitHub Projects, com os itens de `docs/PROXIMOS_PASSOS.md`
Reuniões sem registro escrito	Registrar decisões e impedimentos de cada reunião num arquivo do repositório
Sem ciclos com início e fim definidos	Adotar sprints de uma semana, com revisão das entregas junto aos mentores no fim de cada uma
Trabalho de código concentrado em poucas pessoas	Distribuir tarefas de código com par (uma pessoa revisa o pull request da outra)